**ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH**
**TRƯỜNG ĐẠI HỌC SÀI GÒN**
**KHOA CÔNG NGHỆ THÔNG TIN**

---

**ĐOÀN MINH ĐỨC**
**HOÀNG DŨNG**

# NGHIÊN CỨU HỆ THỐNG TẠO BẢN ĐỒ NGẪU NHIÊN THÍCH NGHI THEO HÀNH VI NGƯỜI CHƠI VÀ CÁC YẾU TỐ GAMEPLAY

**ĐỒ ÁN CHUYÊN NGÀNH (BÁO CÁO GIỮA KỲ)**
NGÀNH: CÔNG NGHỆ THÔNG TIN

**Thành phố Hồ Chí Minh, năm 2026**

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

ĐOÀN MINH ĐỨC
HOÀNG DŨNG

# NGHIÊN CỨU HỆ THỐNG TẠO BẢN ĐỒ NGẪU NHIÊN THÍCH NGHI THEO HÀNH VI NGƯỜI CHƠI VÀ CÁC YẾU TỐ GAMEPLAY

**ĐỒ ÁN CHUYÊN NGÀNH (BÁO CÁO GIỮA KỲ)**
NGÀNH: CÔNG NGHỆ THÔNG TIN

**GIẢNG VIÊN HƯỚNG DẪN:**
**TS. PHÂN TẤN QUỐC**

**Thành phố Hồ Chí Minh, năm 2026**

## LỜI CAM ĐOAN

Chúng tôi tên là Đoàn Minh Đức và Hoàng Dũng, sinh viên ngành Công nghệ Thông tin, xin cam đoan rằng đồ án chuyên nghành với đề tài “Nghiên cứu hệ thống tạo bản đồ ngẫu nhiên thích nghi theo hành vi người chơi và các yếu tố gameplay” là công trình nghiên cứu do chính chúng tôi thực hiện dưới sự hướng dẫn của TS. Phan Tấn Quốc.

Những kết quả nghiên cứu được trình bày trong đồ án là công trình nghiên cứu của riêng tôi, được thực hiện trong quá trình học tập và nghiên cứu tại trường. Các nội dung, số liệu, hình ảnh và kết quả nghiên cứu của những tác giả khác được sử dụng trong đồ án đều đã được trích dẫn đầy đủ và ghi rõ nguồn trong phần tài liệu tham khảo.

Trong quá trình thực hiện đồ án, tôi có tham khảo các tài liệu khoa học liên quan đến lĩnh vực Procedural Content Generation, trong đó có công trình của J. Freiknecht (2021) về *Procedural Content Generation for Games*. Các nội dung tham khảo này đều được sử dụng nhằm mục đích nghiên cứu học thuật và đã được trích dẫn theo đúng quy định.

Tôi xin hoàn toàn chịu trách nhiệm về tính trung thực và chính xác của các nội dung được trình bày trong đồ án. Nếu có bất kỳ sai sót hoặc vi phạm nào liên quan đến bản quyền và học thuật, tôi xin hoàn toàn chịu trách nhiệm trước nhà trường và các quy định hiện hành.

(Ký và ghi rõ họ tên)

Đoàn Minh Đức

Hoàng Dũng

## LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện đồ án chuyên nghành, chúng tôi đã nhận được rất nhiều sự hướng dẫn, giúp đỡ và động viên từ quý thầy cô, gia đình và bạn bè. Với lòng kính trọng và biết ơn sâu sắc, chúng tôi xin gửi lời cảm ơn chân thành đến tất cả những người đã hỗ trợ và tạo điều kiện để chúng tôi hoàn thành đề tài này.

Trước hết, chúng tôi xin gửi lời cảm ơn đến quý thầy cô của khoa Công nghệ Thông tin, Trường Đại học Sài Gòn đã tận tình giảng dạy, truyền đạt những kiến thức quý báu và tạo nền tảng vững chắc cho chúng tôi trong suốt quá trình học tập tại trường.

Đặc biệt, chúng tôi xin bày tỏ lòng biết ơn sâu sắc đến TS. Phan Tấn Quốc, giảng viên hướng dẫn, người đã tận tình hướng dẫn, góp ý và định hướng cho chúng tôi trong suốt quá trình thực hiện đề tài “Nghiên cứu hệ thống tạo bản đồ ngẫu nhiên thích nghi theo hành vi người chơi và các yếu tố gameplay”. Những ý kiến đóng góp và sự hỗ trợ của thầy đã giúp chúng tôi hoàn thiện đề tài một cách tốt hơn.

Chúng tôi cũng xin gửi lời cảm ơn đến gia đình, bạn bè và các anh chị đã luôn động viên, hỗ trợ và tạo điều kiện thuận lợi cho chúng tôi trong quá trình học tập cũng như thực hiện đồ án.

Mặc dù đã cố gắng hoàn thành đề tài với tất cả sự nỗ lực, tuy nhiên do kiến thức và kinh nghiệm còn hạn chế nên đồ án không thể tránh khỏi những thiếu sót. Chúng tôi rất mong nhận được những ý kiến đóng góp quý báu từ quý thầy cô để đề tài được hoàn thiện hơn.

Xin chân thành cảm ơn!

## DANH MỤC CÁC KÝ HIỆU, CÁC CHỮ VIẾT TẮT

| STT | Từ viết tắt | Tiếng anh | Tiếng việt |
|---|---|---|---|
| 1 | PCG | Procedural Content Generation | Tạo nội dung ngẫu nhiên bằng thuật toán. |
| 2 | HP | Health Points | Chỉ số sinh mệnh/lượng máu của nhân vật. |
| 3 | BSP | Binary Space Partitioning | Phân chia không gian nhị phân (thuật toán tạo hầm ngục). |
| 4 | WFC | Wave Function Collapse | Thuật toán sụp đổ hàm sóng |
| 5 | EDPCG | Experience-driven Procedural Content Generation | Tạo nội dung ngẫu nhiên hướng trải nghiệm |
| 6 | DCM | Data Collection Module | Thành phần thu thập dữ liệu |
| 7 | CI | Challenge Index | Chỉ số Thách thức |

# BẢNG DANH MỤC CÁC BẢNG BIỂU

**Bảng 1.1:** So sánh các đặc điểm của 3 phương pháp PCG [1]..... 10

# DANH MỤC CÁC HÌNH ẢNH

**Hình 1.1:** Ví dụ về ứng dụng PCG trong trò chơi .kkrieger, một trò chơi có dung lượng chỉ 96KB nhưng toàn bộ nội dung được tạo bằng thuật toán trong thời gian chạy [1].....7
**Hình 1.2:** Ví dụ về hệ thống tạo nội dung procedural trong Unreal Engine, nơi các phép toán và texture được kết hợp để sinh ra vật liệu một cách tự động [1].....9
**Hình 1.3:** Công cụ sinh và phân bố đối tượng môi trường bằng phương pháp procedural trong các game engine hiện đại [1].....11
**Hình 1.4:** Minh họa quá trình phân chia không gian bằng thuật toán Binary Space Partitioning (BSP) để tạo cấu trúc hầm ngục [1].....12
**Hình 1.5:** Minh họa quá trình tạo môi trường dựa trên các ô gạch và quy tắc ràng buộc, trong đó mỗi ô chỉ có thể kết nối với các ô phù hợp theo quy tắc đã định nghĩa [1].....14
**Hình 1.6**: Minh họa địa hình được tạo bằng hàm nhiễu Perlin Noise, tạo ra sự biến đổi liên tục và tự nhiên của bề mặt địa hình [1].....15
**Hình 2.1:** Kiến trúc tổng thể của hệ thống PCG thích ứng dựa trên hành vi người chơi.....19
**Hình 2.2:** Luồng xử lý dữ liệu trong hệ thống PCG thích ứng.....21
**Hình 2.3:** Mô hình chuyển đổi dữ liệu hành vi người chơi thành chỉ số Challenge Index.....23
**Hình 2.4:** Quy trình điều chỉnh tham số BSP dựa trên Challenge Index.....26

# DANH MỤC CÁC SƠ ĐỒ THUẬT TOÁN

**Thuật giải 1:** Thuật toán Phân chia không gian nhị phân (Binary Space Partitioning - BSP).....11
**Thuật giải 2:** Thuật toán dựa trên quy tắc ô gạch và Wave Function Collapse (WFC).....13

# MỤC LỤC

## Contents

LỜI CAM ĐOAN ..... i
LỜI CẢM ƠN ..... ii
DANH MỤC CÁC KÝ HIỆU, CÁC CHỮ VIẾT TẮT ..... iii
BẢNG DANH MỤC CÁC BẢNG BIỂU ..... iv
DANH MỤC CÁC HÌNH ẢNH ..... v
DANH MỤC CÁC SƠ ĐỒ THUẬT TOÁN ..... vi
MỤC LỤC ..... vii
MỞ ĐẦU ..... 1
1. Lý do chọn đề tài ..... 1
2. Lịch sử nghiên cứu vấn đề ..... 2
3. Mục đích và nhiệm vụ nghiên cứu ..... 3
4. Đối tượng và phạm vi nghiên cứu ..... 3
5. Phương pháp nghiên cứu ..... 4
6. Giả thuyết khoa học ..... 4
7. Những đóng góp mới của đồ án ..... 5
8. Cấu trúc của ĐACN ..... 5
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN VỀ THUẬT

TOÁN TẠO MÔI TRƯỜNG ..... 6
1.1. Tổng quan về tạo nội dung ngẫu nhiên ..... 6
1.1.1. Định nghĩa và vai trò của PCG trong thiết kế trò chơi hiện đại 7
1.1.2. Phân loại các phương pháp PCG ..... 9
1.2. Các thuật toán tạo môi trường phổ biến hiện nay ..... 11
1.2.1. Thuật toán Phân chia không gian nhị phân (Binary Space Partitioning - BSP) ..... 11
1.2.2. Thuật toán dựa trên quy tắc ô gạch và Wave Function Collapse (WFC) ..... 13
1.2.3. Các phương pháp mô phỏng tự nhiên (Cellular Automata, Perlin Noise) ..... 14
1.3. Khái niệm về Experience-driven PCG ..... 15
1.3.1. Mối liên hệ giữa hành vi người dùng và sự thay đổi nội dung ..... 16
1.3.2. Những thách thức trong việc cân bằng giữa tính ngẫu nhiên và trải nghiệm cá nhân hóa ..... 17
1.2. 1.4. Tóm tắt chương ..... 17
CHƯƠNG 2: ĐỀ XUẤT MÔ HÌNH TẠO MÔI TRƯỜNG THÍCH ỨNG DỰA TRÊN HÀNH VI NGƯỜI DÙNG ..... 19
2.1. Kiến trúc hệ thống đề xuất ..... 19
2.1.1. Sơ đồ luồng xử lý dữ liệu ..... 19
2.1.2. Thành phần thu thập dữ liệu ..... 21
2.2. Xây dựng mô hình hành vi người dùng ..... 22

2.2.1. Định nghĩa và chuẩn hóa các chỉ số đầu vào.....23
2.2.2. Cơ chế phân loại trạng thái người chơi.....25
2.3. Giải pháp tích hợp hành vi vào thuật toán PCG.....26
2.3.1. Quy tắc ánh xạ dữ liệu (Mapping Rules).....26
2.3.3. Giải thuật đảm bảo tính khả thi (Playability Guarantee) của bản đồ sau khi điều chỉnh.....30
2.4. Tóm tắt chương.....31
TÀI LIỆU THAM KHẢO.....31

# MỞ ĐẦU

## 1. Lý do chọn đề tài

Trong kỷ nguyên kỹ thuật số hiện nay, ngành công nghiệp trò chơi điện tử (Video Games) đã phát triển vượt bậc, không chỉ về mặt doanh thu mà còn về quy mô và độ phức tạp của nội dung. Người chơi ngày càng kỳ vọng vào những thế giới ảo rộng lớn, chi tiết và có khả năng tương tác cao. Điều này đặt ra một áp lực khổng lồ lên các nhà phát triển game: làm thế nào để tạo ra một khối lượng nội dung đồ sộ (màn chơi, nhiệm vụ, vật phẩm) trong khi vẫn phải đảm bảo ngân sách và thời gian phát hành?

Để giải quyết bài toán này, kỹ thuật Tạo nội dung ngẫu nhiên (Procedural Content Generation - PCG) đã trở thành một công cụ không thể thiếu. Như Jonas Freiknecht đã nhấn mạnh trong công trình nghiên cứu của mình (2021), PCG cho phép chuyển dịch gánh nặng sáng tạo từ con người sang các thuật toán máy tính. Thay vì thiết kế thủ công từng mét vuông của bản đồ, các nhà phát triển xây dựng các hệ thống quy tắc để máy tính tự động sinh ra nội dung.

Tuy nhiên, thách thức lớn nhất của PCG hiện đại không còn là việc "tạo ra nội dung" mà là "tạo ra nội dung có ý nghĩa". Hầu hết các hệ thống PCG hiện nay vẫn hoạt động dựa trên các tham số cố định hoặc ngẫu nhiên đơn thuần, dẫn đến sự thiếu hụt trong việc kết nối với trải nghiệm thực tế của người chơi. Một bản đồ được sinh ra hoàn toàn ngẫu nhiên có thể quá khó đối với một người chơi mới, hoặc quá đơn giản đối với một chuyên gia, làm phá vỡ trạng thái tâm lý hưng phấn (Flow State) – yếu tố then chốt giữ chân người chơi.

Xuất phát từ thực trạng đó, việc nghiên cứu một hệ thống PCG có khả năng "thấu hiểu" hành vi người chơi để tự điều chỉnh môi trường là một hướng đi mang tính cấp thiết. Nếu hệ thống có thể nhận biết người chơi đang gặp khó khăn thông qua lượng máu (HP) bị mất, hay họ đang quá mạnh nhờ trang bị hiện có, để từ đó thay đổi cấu trúc hầm ngục (dungeon) một cách thông minh, trải nghiệm chơi game sẽ trở nên cá nhân hóa và hấp dẫn hơn bao giờ hết. Đó là lý do nhóm chúng tôi thực hiện đề tài: “Nghiên cứu hệ thống tạo bản đồ ngẫu nhiên thích nghi theo hành vi người chơi và các yếu tố gameplay”.

## 2. Lịch sử nghiên cứu vấn đề

Lịch sử của PCG gắn liền với những giới hạn về phần cứng trong những ngày đầu của máy tính.

- **Thập niên 1980:** Các trò chơi kinh điển như *Rogue* (1980) hay *Elite* (1984) đã sử dụng PCG như một giải pháp bắt buộc để nén dữ liệu. Với bộ nhớ cực thấp, việc lưu trữ toàn bộ bản đồ là không thể, do đó các thuật toán sinh ngẫu nhiên dựa trên số hạt giống (seeds) đã ra đời.
- **Thập niên 2000 - 2010:** Sự xuất hiện của các thuật toán như *Perlin Noise* cho địa hình hay *Binary Space Partitioning (BSP)* cho hầm ngục đã giúp nội dung PCG trở nên tự nhiên và có cấu trúc hơn. Các tựa game như *Diablo* hay *Minecraft* đã chứng minh sức mạnh của PCG trong việc tạo ra giá trị chơi lại vô hạn.
- **2015 - Nay:** Các nghiên cứu hiện nay, tiêu biểu là của Jonas Freiknecht (2021) hay Yannakakis & Togelius, bắt đầu tập trung vào khía cạnh "Experience-driven PCG". Nghĩa là PCG không chỉ dừng lại ở thuật toán sinh, mà còn tích hợp các mô hình đánh giá

(Evaluation) và phản hồi (Feedback) để nội dung tạo ra thực sự phục vụ trải nghiệm người dùng.

Mặc dù đã có nhiều nghiên cứu về PCG thích nghi, nhưng việc kết hợp cụ thể các chỉ số hành vi động (real-time metrics) như tốc độ hoàn thành, trang bị và sinh mệnh vào các thuật toán phân chia không gian vẫn còn nhiều dư địa để khai phá, đặc biệt là trong các dòng game Indie với nguồn lực hạn chế.

## 3. Mục đích và nhiệm vụ nghiên cứu

### 3.1. Mục đích nghiên cứu:

Mục đích cốt lõi của đồ án là xây dựng một framework cho phép tạo ra các môi trường hầm ngục (Dungeon) không chỉ ngẫu nhiên về mặt hình học mà còn tương thích về mặt trải nghiệm. Hệ thống sẽ đóng vai trò như một "người quản trò" ảo, tự động cân bằng độ khó thông qua việc thay đổi cấu trúc không gian dựa trên các tham số hành vi của người chơi.

### 3.2. Nhiệm vụ nghiên cứu:

Để đạt được mục đích trên, đồ án tập trung thực hiện các nhiệm vụ sau:

1. Hệ thống hóa lý thuyết về các thuật toán PCG phổ biến (BSP, Cellular Automata) và các mô hình hành vi người chơi.
2. Xây dựng bộ quy tắc ánh xạ (Mapping Rules) để chuyển đổi các chỉ số gameplay (HP, Trang bị, Tốc độ) thành các tham số điều khiển thuật toán sinh bản đồ.
3. Thiết kế và cài đặt một bản mẫu (Prototype) trò chơi 2D sử dụng thuật toán PCG thích nghi.

## 4. Đối tượng và phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** Các thuật toán sinh cấu trúc hầm ngục (Dungeon Generation) và các kỹ thuật thu thập, phân tích dữ liệu hành vi người chơi trong thời gian thực.
- **Phạm vi nghiên cứu:**
    - **Về nội dung:** Tập trung vào loại hình môi trường hầm ngục khép kín, dựa trên ô gạch (Tile-based).
    - **Về hành vi:** Chỉ giới hạn nghiên cứu dựa trên các chỉ số: lượng máu (HP), cấp độ trang bị/kĩ năng, thời gian hoàn thành màn chơi.
    - **Về công nghệ:** Thực hiện trên Game Engine Unity với ngôn ngữ C#, tập trung vào logic thuật toán thay vì chất lượng đồ họa chuyên sâu.

## 5. Phương pháp nghiên cứu

Đồ án sử dụng kết hợp các phương pháp nghiên cứu sau:

- **Phương pháp nghiên cứu lý thuyết:** Phân tích các tài liệu học thuật quốc tế (tiêu biểu là luận án của Jonas Freiknecht) để xây dựng nền tảng về PCG và thiết kế trải nghiệm.
- **Phương pháp phân tích hệ thống:** Chia nhỏ hệ thống thành các module (Module thu thập dữ liệu, Module xử lý logic, Module sinh nội dung) để quản lý độ phức tạp trong quá trình phát triển 2 tháng.

## 6. Giả thuyết khoa học

Giả thuyết của đồ án dựa trên nhận định rằng: Việc tích hợp một lớp điều khiển dựa trên dữ liệu hành vi vào giữa quá trình sinh nội dung

ngẫu nhiên sẽ giúp thu hẹp khoảng cách giữa "sự ngẫu nhiên của máy tính" và "mục đích của người thiết kế". Nếu thành công, hệ thống sẽ giúp giảm tỷ lệ người chơi rời bỏ game do mất cân bằng độ khó, đồng thời tạo ra một môi trường chơi game mang tính tương tác cá nhân hóa cao mà vẫn đảm bảo được hiệu năng xử lý của thiết bị.

## 7. Những đóng góp mới của đồ án

Đồ án tập trung vào việc hiện thực hóa một giải pháp kỹ thuật cụ thể nhằm đưa ra cách tiếp cận riêng đối với chủ đề tạo môi trường trò chơi. Những đóng góp chính của đồ án bao gồm:

**Xây dựng mô hình tương tác động giữa người chơi và môi trường (Experience-driven PCG):** Đồ án thiết lập một cơ chế phản hồi thời gian thực, nơi các chỉ số gameplay cốt lõi (HP, cấp độ trang bị, thời gian hoàn thành) được sử dụng làm biến số đầu vào để điều chỉnh độ khó. Cách tiếp cận này giúp cá nhân hóa trải nghiệm, đảm bảo thử thách của trò chơi luôn tương xứng với năng lực của người chơi tại từng thời điểm.

**Cải tiến thuật toán sinh màn chơi dựa trên cấu trúc phòng (Room-based Algorithm):** Đề xuất giải pháp tối ưu hóa việc phân bổ các thực thể trong trò chơi (quái vật, chướng ngại vật, bẫy) thông qua hệ thống trọng số (Weights). Thay vì sắp đặt ngẫu nhiên thuần túy, thuật toán sẽ tính toán mật độ và vị trí các đối tượng để vừa tạo ra sự đa dạng, vừa đảm bảo tính logic và khả năng di chuyển xuyên suốt của bản đồ.

## 8. Cấu trúc của ĐACN

Đồ án gồm có 03 chương, được sắp xếp theo lộ trình từ nghiên cứu lý thuyết đến đề xuất giải pháp và thực hiện cài đặt:

- **Chương 1. Cơ sở lý thuyết và Tổng quan về thuật toán tạo môi trường:** Mô tả tóm tắt các khái niệm cơ bản về PCG dựa trên các tài liệu chuyên khảo quốc tế. Tập trung phân tích các thuật toán tạo hầm ngục hiện nay và xác định những kẽ hở mà dữ liệu hành vi người chơi có thể can thiệp để thay đổi kết quả sinh bản đồ.
- **Chương 2. Đề xuất mô hình tạo môi trường thích ứng dựa trên hành vi người chơi:** Mô tả tóm tắt cách tiếp cận riêng của nhóm trong việc xây dựng bộ quy tắc ánh xạ dữ liệu. Chương này chi tiết hóa logic của logic thuật toán – cách hệ thống đưa ra quyết định thay đổi hình thái bản đồ dựa trên chỉ số sinh mệnh và sức mạnh của người chơi.
- **Chương 3. Thiết kế thực nghiệm và cài đặt chương trình:** Mô tả tóm tắt quá trình lập trình thực tế trên Unity. Chương này trình bày các lớp đối tượng chính, mã nguồn triển khai thuật toán BSP cải tiến và các hình ảnh kết quả thực nghiệm để chứng minh tính đúng đắn của phương pháp tiếp cận đã đề xuất.

# CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN VỀ THUẬT TOÁN TẠO MÔI TRƯỜNG

## 1.1. Tổng quan về tạo nội dung ngẫu nhiên

Trong quá trình phát triển trò chơi điện tử hiện đại, việc xây dựng các thế giới ảo có quy mô lớn và nội dung phong phú là một thách thức đáng kể đối với các nhà phát triển. Các trò chơi ngày nay thường yêu cầu số lượng lớn các yếu tố môi trường như bản đồ, địa hình, nhiệm vụ, vật phẩm và nhân vật. Nếu toàn bộ nội dung này được thiết kế hoàn toàn

thủ công, chi phí phát triển sẽ tăng lên đáng kể và thời gian sản xuất cũng trở nên kéo dài.

Để giải quyết vấn đề này, các nhà phát triển game đã áp dụng kỹ thuật Tạo nội dung ngẫu nhiên (Procedural Content Generation – PCG). Theo Freiknecht [1], PCG là phương pháp sử dụng các thuật toán và quy tắc được lập trình sẵn để tự động tạo ra nội dung trong trò chơi. Thay vì xây dựng từng yếu tố của môi trường bằng tay, các nhà thiết kế chỉ cần xây dựng một hệ thống sinh nội dung (generator), từ đó máy tính có thể tạo ra nhiều biến thể khác nhau của môi trường trò chơi.

Một trong những đặc điểm quan trọng của PCG là khả năng tạo ra số lượng lớn nội dung từ một tập hợp quy tắc tương đối nhỏ. Nhờ đó, PCG giúp giảm đáng kể khối lượng công việc của nhà thiết kế, đồng thời tăng tính đa dạng và khả năng chơi lại của trò chơi. Nhiều tựa game nổi tiếng như các trò chơi roguelike hay các trò chơi thế giới mở đã tận dụng PCG để tạo ra các bản đồ hoặc môi trường khác nhau trong mỗi lần chơi.

Ngoài ra, PCG còn giúp tối ưu hóa việc sử dụng tài nguyên phần cứng. Thay vì lưu trữ toàn bộ dữ liệu bản đồ trong bộ nhớ, nhiều trò chơi có thể sử dụng thuật toán để sinh nội dung trong thời gian thực khi người chơi khám phá thế giới. Điều này giúp giảm dung lượng cài đặt của trò chơi và cho phép tạo ra các môi trường có quy mô lớn hơn.

**Hình 1.1:** Ví dụ về ứng dụng PCG trong trò chơi .kkrieger, một trò chơi có dung lượng chỉ 96KB nhưng toàn bộ nội dung được tạo bằng thuật toán trong thời gian chạy [1].

### 1.1.1. Định nghĩa và vai trò của PCG trong thiết kế trò chơi hiện đại

#### 1.1.1.1. Định nghĩa về PCG

Dựa trên nghiên cứu của Jonas Freiknecht [1], tạo nội dung ngẫu nhiên (Procedural Content Generation - PCG) được hiểu là việc sử dụng các thuật toán để tạo ra các dữ liệu số trong trò chơi điện tử một cách tự động. Thay vì các nhà thiết kế phải tạo ra mọi tài nguyên thủ công, họ thiết kế ra một "máy tạo" (Generator) có khả năng sản sinh ra vô số biến thể nội dung dựa trên một tập hợp các quy tắc logic.

Một khái niệm quan trọng đi kèm là tính "Pseudorandom" (giả ngẫu nhiên). Thuật toán không tạo ra nội dung hoàn toàn hỗn loạn mà dựa trên một giá trị khởi tạo gọi là "Seed". Theo [1], việc sử dụng cùng một Seed trên cùng một thuật toán sẽ luôn trả về một kết quả nội dung giống hệt nhau, giúp tối ưu hóa việc lưu trữ và tái hiện thế giới game mà không cần lưu dữ liệu thô.

#### 1.1.1.2. Vai trò của PCG trong thiết kế trò chơi hiện đại

Trong kỷ nguyên của các trò chơi có thế giới mở và nội dung khổng lồ, PCG đóng vai trò then chốt:

- **Giải quyết bài toán "Mountain of Content":** Jonas [1] nhấn mạnh rằng khối lượng công việc để tạo ra một thế giới game hiện đại là quá lớn đối với con người. PCG cho phép tạo ra hàng tỷ hành tinh (như trong *No Man's Sky*) hoặc hàng triệu hầm ngục khác nhau chỉ với một đội ngũ nhân sự nhỏ.
- **Mở rộng giá trị chơi lại (Replayability):** Đối với các dòng game như Roguelike (điển hình là *Rogue* hoặc *Soul Knight*), PCG đảm bảo mỗi lần người chơi bắt đầu lại là một trải nghiệm mới. Việc không thể học thuộc lòng bản đồ buộc người chơi phải tập trung vào kỹ năng và phản xạ, giúp kéo dài vòng đời của trò chơi [1].
- **Tối ưu hóa tài nguyên phần cứng:** PCG cho phép sinh nội dung ngay trong thời gian thực (On-the-fly), giúp giảm dung lượng cài đặt của trò chơi vì dữ liệu màn chơi được tính toán bởi CPU thay vì lưu trữ sẵn trong ổ cứng.

### 1.1.2. Phân loại các phương pháp PCG

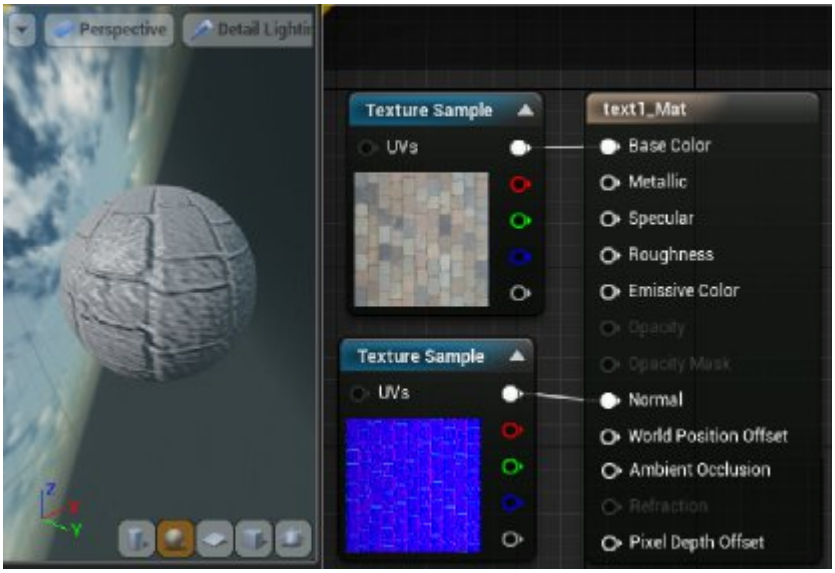
Dựa trên cấu trúc lý thuyết được Jonas Freiknecht tổng hợp và phân tích [1], các phương pháp PCG hiện nay được chia thành ba nhóm chính dựa trên cách tiếp cận kỹ thuật:

#### 1.1.2.1. PCG dựa trên thuật toán (Algorithm-based)

Đây là phương pháp sử dụng các hàm số và cấu trúc dữ liệu thuần túy để xây dựng nội dung.

- **Cơ chế:** Sử dụng các thuật toán như phân chia không gian nhị phân (BSP) hoặc các hàm nhiễu (Perlin Noise).

- **Đặc điểm:** Tốc độ thực thi rất nhanh, phù hợp cho việc tạo ra cấu trúc thô của bản đồ hoặc địa hình tự nhiên. Tuy nhiên, phương pháp này đòi hỏi người lập trình phải kiểm soát chặt chẽ để kết quả không bị rời rạc hoặc phi logic.




**Hình 1.2:** Ví dụ về hệ thống tạo nội dung procedural trong Unreal Engine, nơi các phép toán và texture được kết hợp để sinh ra vật liệu một cách tự động [1].

#### 1.1.2.2. PCG dựa trên ràng buộc (Constraint-based)

Phương pháp này tập trung vào việc thiết lập các điều kiện mà nội dung phải thỏa mãn.

- **Cơ chế:** Thuật toán nổi bật nhất là *Wave Function Collapse* (WFC) hoặc các hệ thống giải quyết ràng buộc (Constraint Satisfaction Problems). Hệ thống sẽ loại bỏ các khả năng vi phạm quy tắc (ví dụ: một ô tường không thể đặt giữa lối đi).

- **Đặc điểm:** Đảm bảo tính nhất quán và thẩm mỹ cực cao cho môi trường game, thường được dùng để tạo ra các kiến trúc phức tạp hoặc trang trí nội thất trong phòng.

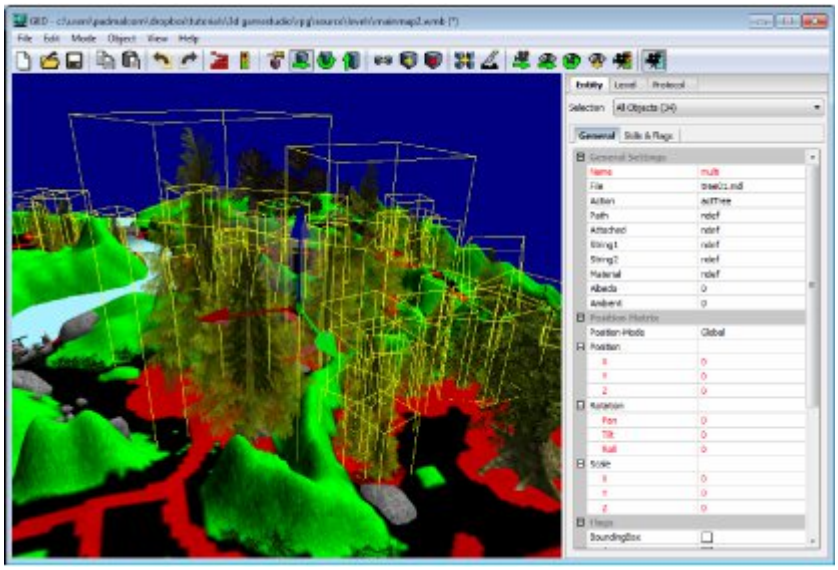
#### 1.1.2.3. PCG dựa trên dữ liệu (Data-driven)

Đây là xu hướng hiện đại, kết hợp sức mạnh của trí tuệ nhân tạo và hành vi người dùng.

- **Cơ chế:** Thuật toán học hỏi từ các mẫu thiết kế sẵn có của con người hoặc dữ liệu thực tế từ quá trình chơi.
- **Đặc điểm:** Nội dung sinh ra mang tính cá nhân hóa cao. Trong đồ án này, việc sử dụng chỉ số HP và trang bị của người chơi để điều chỉnh trọng số (Weights) trong hầm ngục chính là sự kết hợp giữa thuật toán và dữ liệu hành vi.

**Bảng 1.1:** So sánh các đặc điểm của 3 phương pháp PCG [1]

| **Đặc điểm** | **Dựa trên thuật toán** | **Dựa trên ràng buộc** | **Dựa trên dữ liệu** |
|---|---|---|---|
| **Tính kiểm soát** | Thấp | Rất cao | Trung bình |
| **Độ phức tạp** | Thấp | Cao | Rất cao |
| **Ứng dụng điển hình** | Tạo cấu trúc hầm ngục | Xây dựng kiến trúc | Cá nhân hóa trải nghiệm |



**Hình 1.3:** Công cụ sinh và phân bố đối tượng môi trường bằng phương pháp procedural trong các game engine hiện đại [1].

## 1.2. Các thuật toán tạo môi trường phổ biến hiện nay

Trong lĩnh vực tạo nội dung ngẫu nhiên cho trò chơi điện tử, nhiều phương pháp và thuật toán đã được phát triển nhằm tạo ra các môi trường phong phú và đa dạng. Mỗi phương pháp đều có những ưu điểm riêng, phù hợp với từng loại nội dung và mục tiêu thiết kế khác nhau. Theo tổng hợp của Jonas Freiknecht [1], các thuật toán tạo môi trường thường được chia thành ba nhóm chính bao gồm các phương pháp dựa trên cấu trúc không gian, các phương pháp dựa trên quy tắc ô gạch, và các phương pháp mô phỏng tự nhiên.

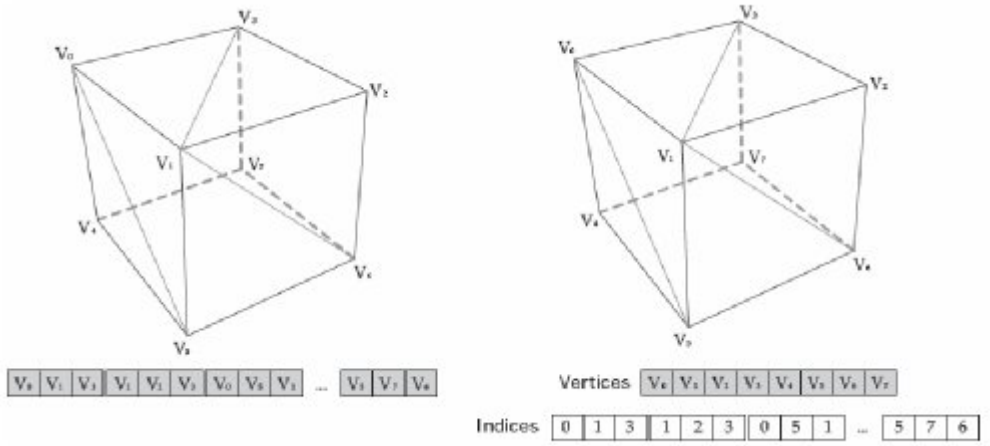
### 1.2.1. Thuật toán Phân chia không gian nhị phân (Binary Space Partitioning - BSP)

**Thuật giải 1:** Thuật toán Phân chia không gian nhị phân (Binary Space Partitioning - BSP)

Binary Space Partitioning (BSP) là một trong những thuật toán phổ biến nhất trong việc tạo bản đồ hầm ngục (Dungeon Generation) cho các trò chơi dạng roguelike. Ý tưởng chính của thuật toán là chia không gian bản đồ thành nhiều vùng nhỏ hơn bằng cách sử dụng các phép phân chia đệ quy theo chiều ngang hoặc chiều dọc.

Cụ thể, thuật toán bắt đầu với một vùng bản đồ lớn. Sau đó vùng này được chia thành hai phần nhỏ hơn bằng một đường cắt ngẫu nhiên theo chiều ngang hoặc chiều dọc. Quá trình này tiếp tục được lặp lại nhiều lần cho đến khi đạt đến kích thước phòng mong muốn. Sau khi các vùng được tạo ra, các phòng sẽ được đặt trong từng vùng và các hành lang sẽ được tạo ra để kết nối các phòng với nhau.

Phương pháp BSP có một số ưu điểm quan trọng. Thứ nhất, nó đảm bảo rằng các phòng được phân bố hợp lý trong không gian bản đồ và tránh việc các phòng bị chồng lấn lên nhau. Thứ hai, cấu trúc phân cấp của cây BSP giúp việc quản lý và truy xuất dữ liệu trở nên dễ dàng hơn. Chính vì vậy, BSP thường được sử dụng trong nhiều trò chơi roguelike hiện đại.




**Hình 1.4:** Minh họa quá trình phân chia không gian bằng thuật toán Binary Space Partitioning (BSP) để tạo cấu trúc hầm ngục [1].

### 1.2.2. Thuật toán dựa trên quy tắc ô gạch và Wave Function Collapse (WFC)

**Thuật giải 2:** Thuật toán dựa trên quy tắc ô gạch và Wave Function Collapse (WFC)

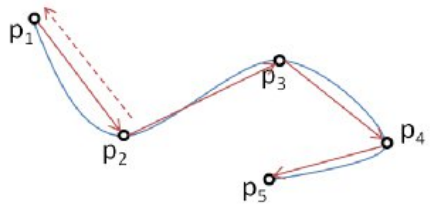
Bên cạnh các phương pháp phân chia không gian, một hướng tiếp cận khác trong tạo môi trường là sử dụng các hệ thống dựa trên quy tắc của ô gạch (Tile-based generation). Trong phương pháp này, bản đồ được tạo thành từ các ô gạch nhỏ, mỗi ô gạch đại diện cho một loại địa hình hoặc cấu trúc nhất định như tường, sàn hoặc cửa.

Một trong những thuật toán nổi bật trong nhóm này là Wave Function Collapse (WFC). Thuật toán này hoạt động dựa trên việc xác định các ràng buộc giữa các ô gạch. Mỗi ô gạch chỉ có thể xuất hiện cạnh những ô gạch phù hợp với quy tắc đã được định nghĩa trước đó.

Quá trình tạo bản đồ bắt đầu bằng việc gán cho mỗi vị trí trên bản đồ một tập hợp các khả năng có thể xảy ra. Sau đó hệ thống sẽ dần dần

loại bỏ những khả năng vi phạm ràng buộc, cho đến khi mỗi vị trí chỉ còn lại một lựa chọn hợp lệ. Cách tiếp cận này giúp đảm bảo rằng bản đồ cuối cùng có cấu trúc hợp lý và thẩm mỹ.

Phương pháp WFC đặc biệt phù hợp cho việc tạo ra các môi trường có tính kiến trúc cao như thành phố, lâu đài hoặc các công trình phức tạp.




**Hình 1.5:** Minh họa quá trình tạo môi trường dựa trên các ô gạch và quy tắc ràng buộc, trong đó mỗi ô chỉ có thể kết nối với các ô phù hợp theo quy tắc đã định nghĩa [1].

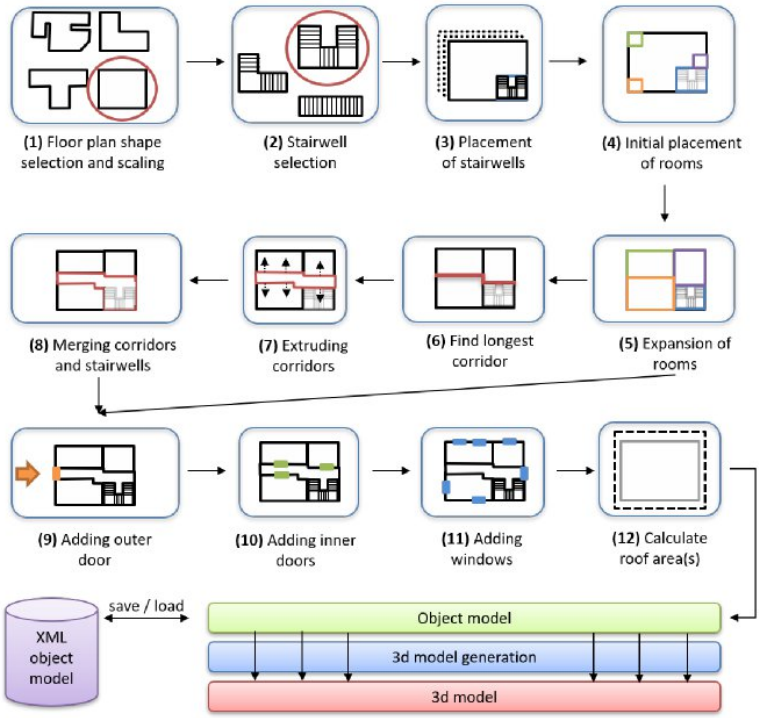
### 1.2.3. Các phương pháp mô phỏng tự nhiên (Cellular Automata, Perlin Noise)

Ngoài các thuật toán dựa trên cấu trúc không gian và quy tắc ô gạch, nhiều phương pháp tạo môi trường còn dựa trên việc mô phỏng các hiện tượng tự nhiên. Hai kỹ thuật phổ biến trong nhóm này là Cellular Automata và Perlin Noise.

Cellular Automata là một mô hình tính toán trong đó bản đồ được biểu diễn dưới dạng một lưới các ô. Trạng thái của mỗi ô sẽ thay đổi theo thời gian dựa trên trạng thái của các ô lân cận. Phương pháp này thường được sử dụng để tạo ra các cấu trúc giống hang động hoặc địa hình tự nhiên.

Trong khi đó, Perlin Noise là một hàm nhiễu được phát triển nhằm tạo ra các biến đổi mượt mà trong không gian. Hàm nhiễu này thường được sử dụng để tạo địa hình, đồi núi hoặc các bề mặt tự nhiên trong trò chơi.

Nhờ khả năng tạo ra các biến đổi tự nhiên và liên tục, Perlin Noise đã trở thành một công cụ quan trọng trong việc tạo địa hình quy mô lớn trong nhiều trò chơi thế giới mở.




**Hình 1.6**: Minh họa địa hình được tạo bằng hàm nhiễu Perlin Noise, tạo ra sự biến đổi liên tục và tự nhiên của bề mặt địa hình [1].

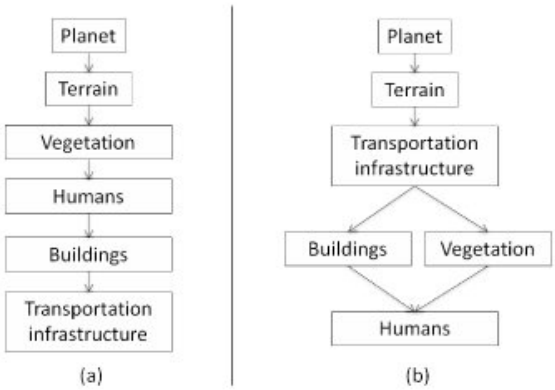
## 1.3. Khái niệm về Experience-driven PCG

Trong những năm gần đây, lĩnh vực Procedural Content Generation đã phát triển theo một hướng mới được gọi là Experience-driven Procedural Content Generation (EDPCG). Khác với các phương pháp PCG truyền thống chỉ tập trung vào việc tạo nội dung ngẫu nhiên,

EDPCG hướng tới việc tạo ra nội dung dựa trên trải nghiệm và hành vi của người chơi.

Theo hướng tiếp cận này, hệ thống sẽ thu thập dữ liệu từ quá trình chơi của người dùng, chẳng hạn như thời gian hoàn thành màn chơi, mức độ sát thương nhận vào, hoặc các lựa chọn chiến thuật của người chơi. Những dữ liệu này sau đó được sử dụng để điều chỉnh các tham số của thuật toán tạo nội dung nhằm mang lại trải nghiệm phù hợp hơn.

Mục tiêu của EDPCG không chỉ là tạo ra nội dung đa dạng mà còn đảm bảo rằng độ khó và cấu trúc của môi trường game có thể thích ứng với từng người chơi khác nhau.




**Hình 1.7:** Sơ đồ tổng quát của hệ thống Procedural Content Generation, trong đó các tham số đầu vào của thuật toán có thể được điều chỉnh để tạo ra các biến thể nội dung khác nhau [1].

### 1.3.1. Mối liên hệ giữa hành vi người dùng và sự thay đổi nội dung

Trong hệ thống EDPCG, hành vi của người chơi đóng vai trò như một nguồn dữ liệu quan trọng giúp hệ thống hiểu được mức độ kỹ năng và phong cách chơi của từng cá nhân.

Ví dụ, nếu hệ thống nhận thấy người chơi thường xuyên hoàn thành màn chơi trong thời gian rất ngắn và ít bị mất máu, điều này có thể cho thấy người chơi đang có trình độ cao hơn so với độ khó hiện tại. Trong trường hợp này, hệ thống có thể tăng độ phức tạp của bản đồ, tăng số lượng kẻ địch, chỉ số kẻ địch, điều thắng, vật cản...

Ngược lại, nếu người chơi thường xuyên thất bại hoặc mất nhiều máu trong quá trình chơi, hệ thống có thể điều chỉnh nội dung theo hướng giảm độ khó để giúp người chơi tiếp tục trải nghiệm trò chơi.

Nhờ cơ chế này, EDPCG cho phép tạo ra một môi trường trò chơi có khả năng thích nghi linh hoạt với từng người chơi.

### 1.3.2. Những thách thức trong việc cân bằng giữa tính ngẫu nhiên và trải nghiệm cá nhân hóa

Mặc dù EDPCG mang lại nhiều lợi ích trong việc cải thiện trải nghiệm người chơi, việc triển khai hệ thống này cũng đặt ra nhiều thách thức.

Một trong những thách thức lớn nhất là cân bằng giữa tính ngẫu nhiên và tính cá nhân hóa của nội dung. Nếu hệ thống quá tập trung vào dữ liệu người chơi, nội dung sinh ra có thể trở nên quá dễ đoán và mất đi yếu tố bất ngờ vốn là đặc trưng của các trò chơi sử dụng PCG.

Ngược lại, nếu mức độ ngẫu nhiên quá cao, hệ thống sẽ không thể đảm bảo rằng nội dung được tạo ra phù hợp với khả năng của người chơi.

Do đó, một hệ thống EDPCG hiệu quả cần phải thiết kế các cơ chế điều chỉnh hợp lý, cho phép kết hợp giữa yếu tố ngẫu nhiên của thuật toán và dữ liệu hành vi của người chơi nhằm tạo ra trải nghiệm cân bằng và hấp dẫn.

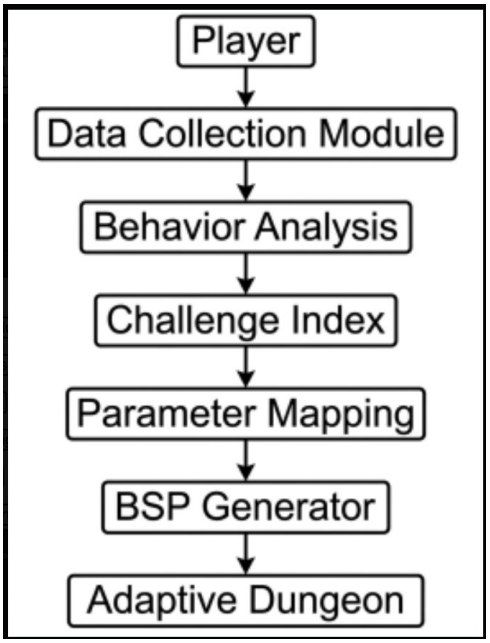
## 1.4. Tóm tắt chương

Chương 1 hệ thống hóa cơ sở lý thuyết về PCG, khẳng định vai trò của thuật toán trong việc tự động hóa sản xuất nội dung và tối ưu hóa trải nghiệm trò chơi. Nội dung trọng tâm đi sâu vào các thuật toán phân chia không gian (BSP), ràng buộc ô gạch (WFC) và mô phỏng tự nhiên để kiến tạo môi trường hầm ngục đa dạng. Đặc biệt, chương giới thiệu mô hình EDPCG – hướng tiếp cận hiện đại cho phép nội dung game thích nghi trực tiếp với hành vi người chơi. Đây là nền tảng quan trọng để thiết lập mối liên hệ giữa các chỉ số gameplay (HP, trang bị, thời gian).

# CHƯƠNG 2: ĐỀ XUẤT MÔ HÌNH TẠO MÔI TRƯỜNG THÍCH ỨNG DỰA TRÊN HÀNH VI NGƯỜI DÙNG

## 2.1. Kiến trúc hệ thống đề xuất

Trong thiết kế trò chơi truyền thống, việc tạo nội dung ngẫu nhiên (PCG) thường được thực hiện theo cơ chế "mở", nghĩa là thuật toán sinh ra nội dung dựa trên các tham số cố định mà không quan tâm đến kết quả trải nghiệm của người chơi. Để khắc phục sự thiếu hụt này, hệ thống đề xuất trong đồ án chuyển dịch sang mô hình Hệ thống điều khiển phản hồi vòng kín (Closed-loop Feedback System).

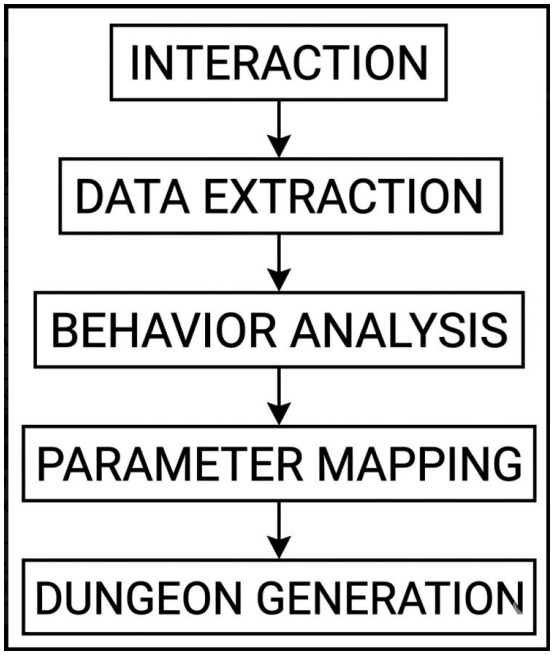



**Hình 2.1:** Kiến trúc tổng thể của hệ thống PCG thích ứng dựa trên hành vi người chơi.

### 2.1.1. Sơ đồ luồng xử lý dữ liệu

Kiến trúc hệ thống được chia thành một chu trình khép kín gồm bốn thực thể tương tác liên tục: Người chơi (Agent), Bộ thu thập (Observer), Bộ xử lý logic (Logic Processor) và Bộ sinh nội dung (Generator).

- **Giai đoạn Tương tác (Interaction Phase):** Người chơi thực hiện các hành động trong môi trường hầm ngục (di chuyển, chiến đấu, nhặt vật phẩm). Tại đây, mọi biến số về trạng thái nhân vật đều thay đổi liên tục. Hệ thống không can thiệp vào giai đoạn này để đảm bảo tính công bằng và nhất quán của màn chơi hiện tại.
- **Giai đoạn Trích xuất & Chuyển hóa (Extraction & Transformation):** Ngay khi người chơi hoàn thành màn chơi (thông qua cổng dịch chuyển), dữ liệu thô từ bộ nhớ game sẽ được trích xuất. Các dữ liệu này không được đưa trực tiếp vào thuật toán mà phải đi qua một bước "Trung gian hóa". Bước này giúp lọc bỏ các nhiễu thông tin và đưa dữ liệu về dạng số học chuẩn hóa.
- **Giai đoạn Ánh xạ Tham số (Parameter Mapping):** Đây là "bộ não" của hệ thống đề xuất. Dựa trên kết quả phân tích hành vi, bộ xử lý sẽ đưa ra các quyết định thay đổi tham số cho thuật toán BSP. Thay vì thay đổi bản đồ một cách ngẫu nhiên, hệ thống thực hiện thay đổi có mục đích: nếu người chơi quá mạnh, hệ thống sẽ "siết chặt" các quy tắc sinh phòng; nếu người chơi yếu, hệ thống sẽ "nới lỏng" các ràng buộc không gian.
- **Giai đoạn Tái cấu trúc (Reconstruction):** Thuật toán BSP nhận các tham số đã điều chỉnh để tiến hành chia cắt không gian. Kết quả là một hầm ngục mới được hình thành, mang trong mình đặc tính thử thách đã được tùy biến riêng cho năng lực của người chơi vừa thực hiện.




**Hình 2.2:** Luồng xử lý dữ liệu trong hệ thống PCG thích ứng.

### 2.1.2. Thành phần thu thập dữ liệu

Thành phần thu thập dữ liệu đóng vai trò là lớp giao tiếp giữa hệ thống gameplay và bộ tạo nội dung. Để đảm bảo tính tối giản và hiệu quả cho thuật toán BSP thích ứng, hệ thống chỉ tập trung thu thập ba nhóm chỉ số cốt lõi. Các chỉ số này được trích xuất tại thời điểm người chơi hoàn thành một màn chơi (Stage) để làm dữ liệu đầu vào cho màn chơi kế tiếp.

#### 2.1.2.1. Chỉ số sinh mệnh

Đây là thông số trực quan nhất để đo lường độ khó của môi trường đối với năng lực phòng thủ của người chơi. Hệ thống sẽ ghi lại lượng máu còn lại của nhân vật tại thời điểm chạm vào cổng dịch chuyển kết thúc màn. Chỉ số này cho biết liệu người chơi đã vượt qua thử thách một cách dễ dàng hay đã phải chật vật để sinh tồn.

#### 2.1.2.2. Chỉ số hiệu suất thời gian

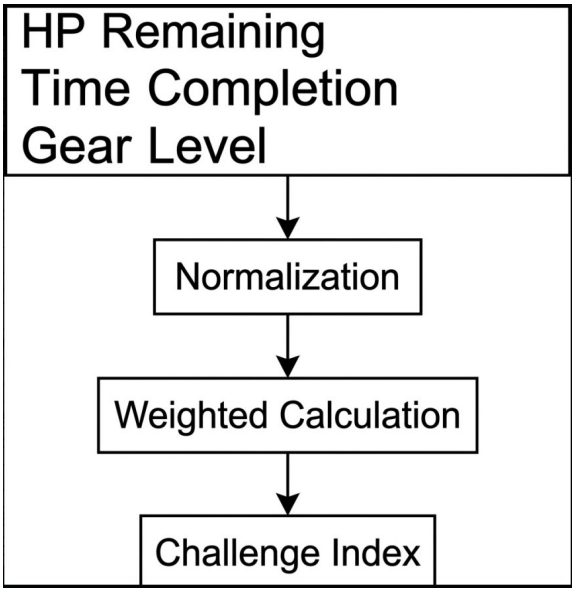
Hệ thống ghi lại tổng thời gian (tính bằng giây) kể từ khi người chơi xuất hiện tại phòng đầu tiên cho đến khi tiêu diệt hết mục tiêu và di chuyển đến lối thoát. Chỉ số này không chỉ phản ánh tốc độ di chuyển mà còn cho thấy khả năng xử lý các vật cản và quái vật trong không gian hầm ngục. Thời gian thực tế sẽ được so sánh với mốc thời gian tiêu chuẩn của mỗi cấp độ để đánh giá mức độ thuần thục của người chơi.

#### 2.1.2.3. Chỉ số sức mạnh trang bị

Thay vì thu thập toàn bộ kho đồ, hệ thống chỉ tập trung vào cấp độ sức mạnh tổng hợp (Gear Level) của vũ khí, kĩ năng Chỉ số này giúp hệ thống dự đoán khả năng gây sát thương của người chơi, từ đó điều chỉnh mật độ vật cản hoặc độ phức tạp của các phòng trong thuật toán BSP nhằm tạo ra những không gian giao tranh phù hợp với sức mạnh hiện có.

## 2.2. Xây dựng mô hình hành vi người dùng

Mô hình hành vi người dùng đóng vai trò là bộ lọc thông tin, giúp chuyển đổi các tương tác vật lý và các trạng thái nhân vật trong game thành các giá trị định lượng. Mục tiêu của mô hình này là xác định xem người chơi đang ở trạng thái nào: đang quá nhàn rỗi vì game quá dễ, hay đang gặp căng thẳng vì độ khó quá cao.




**Hình 2.3:** Mô hình chuyển đổi dữ liệu hành vi người chơi thành chỉ số Challenge Index.

### 2.2.1. Định nghĩa và chuẩn hóa các chỉ số đầu vào

Dữ liệu thô thu thập từ Module DCM (Data Collection Module) thường có các đơn vị đo lường khác nhau (giây, điểm, cấp độ). Để đưa chúng vào một công thức duy nhất, ta cần thực hiện bước Chuẩn hóa (Normalization) về thang đo [0, 1]

**Chú thích về hàm Clamp:** Clamp(value, min, max) là một hàm toán học dùng để giới hạn một giá trị trong một khoảng nhất định. Nếu giá trị nhỏ hơn min, nó sẽ trả về min. Nếu lớn hơn max, nó sẽ trả về max. Việc sử dụng clamp trong PCG cực kỳ quan trọng vì nó ngăn chặn việc các chỉ số vượt quá giới hạn (ví dụ: máu ảo hoặc lỗi thời gian) làm sai lệch công thức ánh xạ.

#### 2.2.1.1. Chuẩn hóa Chỉ số Sinh mệnh

Chỉ số này phản ánh năng lực sinh tồn và quản lý rủi ro của người chơi.

$$I_{HP} = \text{clamp}\left(\frac{HP_{current}}{HP_{max}}, 0, 1\right)$$

- **Giải thích:**
  - $HP_{current}$: Lượng máu hiện tại của người chơi khi kết thúc màn.
  - $HP_{max}$: Lượng máu tối đa của nhân vật.
- **Ý nghĩa:** Nếu người chơi kết thúc màn với lượng máu đầy **($I_{HP} = 1$)**, điều này chứng tỏ thử thách hiện tại chưa đủ gây áp lực lên họ.

#### *2.2.1.2. Chuẩn hóa Chỉ số Hiệu suất Thời gian*

Chỉ số thời gian phản ánh tốc độ xử lý tình huống và kỹ năng định hướng không gian.

$$I_{Time} = \text{clamp}\left(\frac{T_{std}}{T_{actual}}, 0, 1\right)$$

- **Giải thích:**
  - $T_{actual}$: Thời gian thực tế người chơi dùng để hoàn thành màn.
  - $T_{std}$: Thời gian tiêu chuẩn dự kiến cho kích thước map đó (ví dụ: 60 giây).
- **Ý nghĩa:** Nếu người chơi vượt màn nhanh hơn thời gian tiêu chuẩn **($T_{actual} < T_{std}$)** tỷ số sẽ lớn hơn 1, nhưng hàm clamp sẽ giữ nó ở mức 1. Điều này cho thấy kỹ năng của người chơi đã vượt ngưỡng yêu cầu.

#### 2.2.1.3. Chuẩn hóa Chỉ số Sức mạnh Trang bị

Chỉ số này đại diện cho tiềm năng chiến đấu dựa trên tài nguyên mà người chơi tích lũy được.

$$I_{Gear} = \textbf{clamp}\left(\frac{Level_{current}}{Level_{cap}}, 0, 1\right)$$

- **Giải thích:**
  - $Level_{current}$: Cấp độ trang bị hiện tại.
  - $Level_{cap}$: Cấp độ tối đa cho phép tại Stage hiện tại.
- **Ý nghĩa:** Một người chơi có trang bị mạnh **($I_{Gear} \approx 1$)** cần một môi trường phức tạp hơn (nhiều vật cản, mê cung sâu hơn) để không cảm thấy nhàm chán.

### 2.2.2. Cơ chế phân loại trạng thái người chơi

Sau khi có các chỉ số thành phần đã chuẩn hóa, hệ thống sẽ tổng hợp chúng lại thành một biến số duy nhất gọi là Chỉ số Thách thức (Challenge Index - CI). Đây là linh hồn của hệ thống PCG thích ứng.

Công thức tính CI được xây dựng theo phương pháp **Trung bình trọng số (Weighted Average)**:

$$CI = (w_{HP} \cdot I_{HP}) + (w_{Time} \cdot I_{Time}) + (w_{Gear} \cdot I_{Gear})$$

**Trong đó:**

- $w_{HP}, w_{Time}, w_{Gear}$ là các Trọng số (Weights). Tổng các trọng số này phải bằng 1. Việc điều chỉnh các trọng số này sẽ làm thay đổi "tính cách" của hệ thống PCG:
  - Nếu ưu tiên sự sinh tồn: Đặt $w_{HP}$ cao (ví dụ: 0.5).
  - Nếu ưu tiên tốc độ hành động: Đặt $w_{Time}$ cao (ví dụ: 0.5).

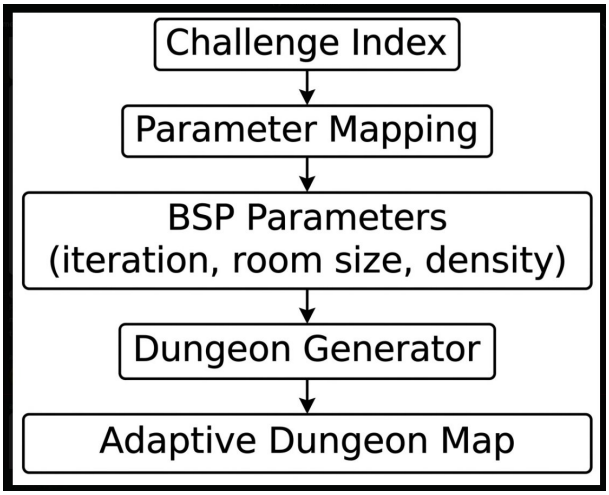
**Phân loại trạng thái dựa trên CI:**

Dựa trên giá trị CI thu được, hệ thống sẽ phân loại người chơi vào các nhóm trạng thái để chuẩn bị cho việc điều chỉnh thuật toán BSP:

1. **Trạng thái Quá tải (CI < 0.3):** Người chơi mất nhiều máu, vượt màn chậm. Hệ thống nhận diện người chơi đang gặp khó khăn. Hành động: Giảm độ phức tạp của hầm ngục tiếp theo.
2. **Trạng thái Cân bằng ($0.3 \leq CI \leq 0.7$):** Độ khó hiện tại phù hợp với kỹ năng người chơi. Đây là vùng lý tưởng để duy trì sự hưng phấn (Flow State). Hành động: Giữ nguyên hoặc tăng nhẹ độ phức tạp.
3. **Trạng thái Nhàm chán ($CI > 0.7$):** Người chơi vượt màn quá dễ dàng với chỉ số tuyệt đối. Hành động: Tăng mạnh số lần chia phòng (Iteration) và mật độ vật cản để tạo thử thách mới.

### 2.3. Giải pháp tích hợp hành vi vào thuật toán PCG

Mục tiêu của giải pháp này là biến Chỉ số Thách thức (CI) từ một giá trị toán học trừu tượng thành các lệnh điều khiển trực tiếp cấu trúc không gian của hầm ngục. Quá trình này được thực hiện thông qua việc can thiệp vào các tham số đầu vào của thuật toán BSP (Binary Space Partitioning) trước khi quá trình phân chia vùng bắt đầu.




**Hình 2.4:** Quy trình điều chỉnh tham số BSP dựa trên Challenge Index.

### 2.3.1. Quy tắc ánh xạ dữ liệu (Mapping Rules)

Quy tắc ánh xạ sử dụng hàm nội suy tuyến tính (Linear Interpolation - Lerp) làm cốt lõi. Hàm này cho phép hệ thống tính toán một giá trị trung gian giữa hai mốc giới hạn dựa trên tỷ lệ của CI.

#### 2.3.1.1. Công thức ánh xạ thuận: Áp dụng cho Số lần lặp và Mật độ vật cản

Đối với các tham số mà khi giá trị của chúng tăng lên thì độ khó của game cũng tăng theo (như số lượng phòng hoặc số lượng vật cản), ta sử dụng công thức ánh xạ thuận:

$$P_{active} = P_{min} + (P_{max} - P_{min}) \times CI$$

- **Giải thích các đại lượng:**
  - $P_{active}$: Giá trị tham số thực tế sẽ nạp vào thuật toán (Ví dụ: maxIteration).
  - $P_{min}$:Giá trị tối thiểu an toàn để thuật toán không bị lỗi (Ví dụ: 3 lần lặp).

- $P_{max}$: Giá trị tối đa để đảm bảo hiệu suất phần cứng (Ví dụ: 7 lần lặp).
- CI: Chỉ số thách thức tính từ mô hình hành vi ở mục 2.2 (giá trị từ 0 đến 1).

- **Chú thích logic:** Khi người chơi có kỹ năng cao **($CI \to 1$)**, giá trị $P_{active}$ sẽ tiến gần về mốc $P_{max}$. Điều này đồng nghĩa với việc hầm ngục sẽ được chia nhỏ thành nhiều phân khu hơn, tạo ra cấu trúc phức tạp.

#### 2.3.1.2. Công thức ánh xạ nghịch: Áp dụng cho Kích thước phòng

Đối với các tham số mà độ khó tăng lên khi giá trị giảm xuống (phòng càng nhỏ thì càng khó di chuyển và chiến đấu), ta sử dụng công thức ánh xạ nghịch:

$$S_{active} = S_{max} - (S_{max} - S_{min}) \times CI$$

- **Giải thích các đại lượng:**
  - $S_{active}$: Kích thước phòng tối thiểu thực tế (minRoomSize).
  - $S_{max}$: Kích thước phòng lớn nhất (tạo không gian dễ dàng cho người chơi yếu).
  - $S_{min}$: Kích thước phòng nhỏ nhất (tăng áp lực chiến đấu cho người chơi giỏi).
- **Chú thích logic:** Phép trừ $(S_{max} - S_{min}) \times CI$ đóng vai trò là "lượng cắt giảm". Nếu người chơi càng giỏi, lượng cắt giảm càng lớn, làm cho kích thước phòng thực tế thu hẹp lại, tạo ra các "nút thắt cổ chai" (bottleneck) trong bản đồ.

### 2.3.2. Điều chỉnh cấu trúc hầm ngục dựa trên trạng thái người chơi

Trong giai đoạn thiết kế lý thuyết, cấu trúc hầm ngục được xem xét như một hệ thống mạng lưới các nút (phòng) và các cạnh (hành lang). Việc điều chỉnh dựa trên Chỉ số Thách thức (CI) sẽ làm thay đổi bản chất hình học và tính chất kết nối của mạng lưới này.

#### 2.3.2.1. Biến đổi hình thái học không gian

Dưới tác động của mô hình thích ứng, hầm ngục sẽ không còn một hình dạng cố định mà biến thiên theo hai thái cực dựa trên năng lực của đối tượng trải nghiệm:

Cấu trúc Đa phân nhánh (Phức tạp hóa): Khi giá trị CI tiệm cận về mức tối đa **(**$CI \approx 1$**)**, lý thuyết BSP sẽ thực thi các nhát cắt liên tục với mật độ dày đặc. Điều này tạo ra một "Hình thái" dạng lưới phức tạp với nhiều nút lá (leaf nodes). Trong kiến trúc trò chơi, việc có quá nhiều phòng nhỏ liên tiếp sẽ làm gia tăng "độ nhiễu" về mặt định hướng, buộc người chơi phải liên tục đưa ra các quyết định rẽ trái hoặc phải, từ đó kéo dài thời gian trải nghiệm và tăng độ khó về mặt tư duy không gian.

- **Cấu trúc Đại sảnh (Đơn giản hóa):** Ngược lại, khi CI ở mức thấp **(**$CI < 0.3$**)**, hệ thống lý thuyết sẽ hạn chế các nhát cắt không gian. Hầm ngục lúc này mang hình thái của các "Đại sảnh" – những phòng có diện tích lớn với cấu trúc ít ngõ cụt. Về mặt lý thuyết, không gian rộng giúp giảm thiểu sự va chạm và cung cấp tầm nhìn bao quát (Line of Sight), cho phép người chơi dễ dàng nhận diện nguy hiểm từ xa khi đang trong trạng thái sinh mệnh thấp.

#### 2.3.2.2. Chiến lược phân bổ mật độ chướng ngại vật lý thuyết

Sự thích ứng không chỉ dừng lại ở khung xương bản đồ (phòng và hành lang) mà còn tác động đến mật độ vật chất bên trong:

- **Mô hình "Nút thắt cổ chai" (Bottleneck Design):** Đối với trạng thái người chơi xuất sắc, mô hình đề xuất việc tăng mật độ vật cản theo các cụm (clusters). Việc rải vật cản tập trung gần các cửa ra vào sẽ tạo ra các điểm thắt chiến thuật. Điều này yêu cầu người chơi phải có kỹ năng di chuyển chính xác trong không gian hẹp.
- **Mô hình "Hành lang an toàn" (Buffer Zone):** Đối với người chơi đang gặp khó khăn, lý thuyết thiết kế sẽ ưu tiên giải phóng không gian xung quanh các lối đi. Việc giảm mật độ vật chất tại trung tâm phòng tạo ra một vùng đệm an toàn, giúp người chơi có thể tự do di chuyển mà không bị cản trở bởi các yếu tố môi trường không mong muốn.

#### 2.3.2.3. Tác động của chỉ số sức mạnh đến cấu trúc phòng

Dựa trên chỉ số trang bị **($I_{Gear}$)**, lý thuyết về cấu trúc hầm ngục sẽ điều chỉnh tỷ lệ giữa chiều dài và chiều rộng (Aspect Ratio) của các phòng. Một người chơi có vũ khí tầm xa mạnh mẽ sẽ gặp thử thách hơn trong những căn phòng hẹp và dài, nơi lợi thế về khoảng cách bị triệt tiêu. Ngược lại, những phòng có hình dạng vuông vức sẽ được ưu tiên cho người chơi có chỉ số trang bị thấp để họ dễ dàng thực hiện các chiến thuật phòng thủ cơ bản.

### 2.3.3. Giải thuật đảm bảo tính khả thi (Playability Guarantee) của bản đồ sau khi điều chỉnh

Vì việc thay đổi các tham số *Iteration* và *Density* ở mức lý thuyết có thể dẫn đến rủi ro tạo ra những bản đồ không thể vượt qua (impossible maps), hệ thống cần thiết lập các quy tắc đảm bảo tính khả thi.

#### 2.3.3.1. Nguyên lý kết nối tối thiểu

Mọi phòng được sinh ra từ quá trình phân chia không gian thích ứng đều phải tuân thủ lý thuyết đồ thị liên thông. Hệ thống giả định rằng mỗi trung tâm của một vùng không gian (Node) phải có ít nhất một đường dẫn (Edge) nối tới các nút lân cận. Quy tắc này đảm bảo rằng dù hầm ngục có bị chia nhỏ đến mức nào, tính kết nối toàn cục vẫn được duy trì.

#### 2.3.3.2. Thuật toán kiểm soát tính thông suốt

Đây là phương pháp lý thuyết được đề xuất để kiểm tra sau khi môi trường đã được rải vật cản.

- **Cơ chế vận hành:** Thuật toán bắt đầu từ một điểm gốc (Start) và "loang" dần ra các không gian trống liền kề.
- **Điều kiện khả thi:** Nếu vùng loang không thể chạm tới điểm đích (Exit) do mật độ vật cản quá dày hoặc cấu trúc chia phòng bị lỗi, hệ thống sẽ thực hiện một "nhát cắt logic" cưỡng bức. Về mặt lý thuyết, đây là quá trình loại bỏ các phần tử ngăn cách trên đường thẳng ngắn nhất nối giữa hai điểm mục tiêu để khai thông lối đi.

#### 2.3.3.3. Ràng buộc giới hạn an toàn (Boundary Constraints)

Để đảm bảo bản đồ thích ứng nhưng vẫn thực dụng, các tham số luôn được khóa trong một dải giá trị an toàn (Safety Bounds). Ví dụ, dù CI có cao đến mức nào, kích thước phòng tối thiểu không bao giờ được nhỏ

hơn kích thước vật lý của thực thể người chơi, đảm bảo không xảy ra hiện tượng kẹt cứng không gian ngay khi bắt đầu.

## 2.4. Tóm tắt chương

Chương 2 đề xuất mô hình tạo hầm ngục thích ứng dựa trên hệ thống phản hồi vòng kín, giúp điều chỉnh tham số thuật toán PCG từ dữ liệu hành vi thực tế của người chơi. Thông qua việc thu thập và chuẩn hóa các chỉ số về sinh mệnh **($I_{HP}$)**, thời gian hoàn thành **($I_{Time}$)** và sức mạnh trang bị **($I_{Gear}$)**, hệ thống xác lập Chỉ số Thách thức **(CI)** để phân loại trạng thái trải nghiệm. Từ giá trị CI này, các quy tắc ánh xạ nội suy tuyến tính sẽ can thiệp vào thuật toán BSP để thay đổi linh hoạt số lần chia phòng, kích thước không gian và mật độ vật cản. Để đảm bảo tính khả thi, hệ thống tích hợp các giải thuật kiểm tra liên thông và loang vùng (Flood Fill), giúp hầm ngục luôn đảm bảo tính kết nối và có lối thoát dù độ khó được đẩy lên cao.

## TÀI LIỆU THAM KHẢO

[1]. J. Freiknecht, *Procedural Content Generation for Games*, Doctoral dissertation, University of Mannheim, 2021.

[2]. G. N. Yannakakis and J. Togelius, "Experience-Driven Procedural Content Generation", *IEEE Transactions on Affective Computing*, vol. 2, no. 3, pp. 147-161, 2011.